UniverBot – ChatBot Builder

1. Problem Statement:

Modern organizations need AI chatbots that answer accurately from their own content, not from generic internet-scale model memory. However, building such a system typically requires combining many difficult components: document ingestion, website crawling, chunking, embeddings, vector search, prompt orchestration, session handling, frontend chat UI, deployment, and security. This complexity creates a major gap between what businesses need and what most teams can realistically build and maintain.

UniverBot addresses this gap by solving a core systems problem:

A user-friendly, multi-tenant platform is needed where non-expert users can create, train, customize, and deploy domain-specific chatbots that provide grounded responses from organization-owned knowledge sources, while remaining scalable, secure, and cost-efficient under free-tier cloud constraints.

2. Introduction:

UniverBot is a full-stack AI chatbot builder platform designed to let users create, train, customize, and deploy chatbots using their own knowledge sources. The project is implemented as a multi-service system with a web dashboard, a backend API, a separate embedding service, and a PostgreSQL database layer with vector support. The central idea is to move from generic chatbot behavior to controlled, source-grounded responses by combining knowledge ingestion, retrieval, and structured conversation execution inside one unified platform.

At a product level, UniverBot supports two main user journeys. The first is the creator journey: a user signs in, creates a bot, defines behavior through prompt and flow configuration, uploads files or scrapes websites, and customizes appearance. The second is the end-user journey: a visitor interacts with the deployed bot through an embeddable chat widget on any website. These two journeys are connected by backend services that handle authentication, knowledge processing, retrieval, and response generation.

From an architecture perspective, the platform is intentionally separated into responsibilities:

1. A frontend application for management and configuration.
2. A backend service for orchestration and API logic.
3. A standalone embedding service for vector generation.
4. A Supabase-backed persistence layer for bot data, documents, vectors, chat logs, and session state.

3. System Topology:

UniverBot follows a multi-service topology with clear separation between management, runtime chat, vector generation, and persistence. The deployed system is organized into five main nodes:

1. Dashboard Web App (Next.js frontend).
2. Embedded Chat Widget (plain JavaScript runtime for third-party websites).
3. Main API Backend (FastAPI orchestration layer).
4. Embedding Microservice (FastAPI + sentence-transformers inference).
5. Supabase (PostgreSQL + pgvector + auth + storage).

```
+----------------------------+
                    |     Creator/Admin User      |
                    +--------------+--------------+
                                   |
                                   v
                    +----------------------------+
                    |   Next.js Dashboard App     |
                    | (bot create/train/settings) |
                    +--------------+--------------+
                                   |
                                   | JWT-authenticated API calls
                                   v
+------------------+   API Key    +----------------------------+    HTTP    +--------------------------+
| Website Visitor  +-------------> |     FastAPI Main Backend    +----------> | Embedding Microservice    |
| + Embedded Widget|               |  (/bot, /chat, /health)     |            | (/embed, /embed/batch)    |
+------------------+               +--------------+--------------+            +--------------------------+
                                                  |
                                                  | SQL + auth
                                                  v
                                    +-----------------------------+
                                    |      Supabase (Postgres)     |
                                    | profiles, bots, documents,   |
                                    | document_chunks (vector),    |
                                    | chat_messages, chat_sessions |
                                    +-----------------------------+# Embedding Service Integration Guide
```

4. System design

The system is designed to let a user create and deploy a custom AI chatbot that answers from user-provided knowledge sources (documents and websites), while supporting configurable conversation flow, session continuity, and embeddable website integration.

The design combines:

1. A management application for bot owners.
2. A runtime chat interface for end users.
3. A backend orchestration API.
4. A dedicated embedding inference service.
5. A PostgreSQL + vector persistence layer.

```
Bot Owner (Dashboard) ------------------------------+
                                                     |
Website Visitor (Embedded Widget) ------------------+--> Main Backend API
                                                         |
                                                         +--> Embedding Microservice
                                                         |
                                                         +--> Supabase/PostgreSQL (pgvector)
```

5. Backend Architectural Layers

The backend is designed as a layered architecture where each layer has a focused responsibility and a controlled dependency direction. The top layer handles HTTP communication, the middle layers enforce security and business orchestration, and the bottom layers manage external integrations and persistence. This structure reduces coupling, improves maintainability, and makes the system easier to scale and test.

At runtime, every request enters through the API layer, passes through authentication and validation, and is then routed into orchestration services such as chat execution, retrieval, embedding generation, scraping, or training. The orchestration layer does not directly expose transport details; instead, it coordinates reusable services. Those services communicate with the database and external components such as the embedding microservice. This separation is what allows the system to support both dashboard control operations and public widget chat operations safely.

Backend Layer Topology

Client Request
-> API Routing Layer
-> Security and Access Control Layer
-> Validation and Contract Layer
-> Orchestration and Business Logic Layer
-> Integration Layer (Embedding, Crawling, LLM, Retrieval)
-> Data Access Layer
-> Persistence Layer (PostgreSQL with vector storage)

Layer Description Table

Layer	Main Role	What It Handles	Why It Exists
API Routing Layer	HTTP entrypoint	Endpoint grouping for health, bot management, and chat	Keeps transport concerns separate from business logic
Security Layer	Access control	User authentication, bot key verification, ownership checks	Prevents unauthorized data access across tenants
Validation Layer	Data contracts	Request and response schemas, flow structure validation	Ensures consistent and safe input/output handling
Orchestration Layer	Runtime coordination	Chat flow execution, retrieval decisions, session progression	Central place for platform behavior and policy
Integration Layer	External capability bridge	Embedding calls, scraping/crawling, response generation provider calls	Isolates third-party or heavy services from core routing
Data Access Layer	Database interaction	Table operations, retrieval queries, session and message persistence	Prevents DB logic from spreading across API handlers
Persistence Layer	Durable state	Bots, documents, chunks, embeddings, sessions, messages	Maintains long-term platform memory and retrieval corpus

How the Layers Work Together in Chat Runtime

A chat request first reaches the API routing layer, where request metadata and authentication credentials are accepted. The security layer validates whether the request is from an authenticated owner or from a widget using a valid bot key. The validation layer ensures payload integrity, including message shape and session identifiers. The orchestration layer then decides whether the bot should follow a configured conversation flow or fallback retrieval path. If retrieval is needed, the integration layer obtains embeddings and executes retrieval logic, after which response generation is performed. Finally, the data access layer writes user and assistant messages and updates session state so that subsequent turns continue correctly.

How the Layers Work Together in Knowledge Ingestion

During document upload or website ingestion, the API layer receives the input and delegates ownership checks to the security layer. After validation and limit checks, the orchestration layer invokes document processing to normalize and

chunk content. The integration layer generates embeddings for each chunk and returns vectors for storage. The data access layer persists metadata, chunks, and embeddings in the persistence layer. This same structure is reused for both file-based and web-based ingestion pipelines, which keeps the ingestion architecture consistent.

Architectural Strength

This layered backend design gives three practical advantages. First, it supports modular evolution, so retrieval, embedding, or flow behavior can be improved without rewriting endpoint code. Second, it enables operational resilience by isolating external-service dependencies. Third, it keeps security boundaries explicit by separating owner operations from public chat runtime access.

Technical Component Map

Backend Component	Core Technical Function	Runtime Behavior
FastAPI app shell	Service bootstrapping and middleware	Initializes app lifecycle, CORS, and router groups
Auth module	Trust boundary enforcement	Verifies owner JWT for management APIs and bot API key for widget chat
Bot API module	Control-plane orchestration	Handles bot CRUD, upload, scrape, crawl, storage stats, key regeneration
Chat API module	Runtime-plane orchestration	Handles session-aware chat, flow routing, fallback retrieval path
Document processor	Knowledge normalization	Extracts text from PDF/DOCX/TXT/MD and creates overlapped chunks
Embedding service client	Vector generation integration	Calls embedding microservice first, falls back to local model on failure
RAG module	Retrieval engine	Executes vector retrieval with lexical fallback and result fusion
Flow executor	Stateful conversation engine	Executes node graph, tracks session node state, builds contextual queries
LLM module	Final response generation	Uses configured Groq client pool with key failover behavior

Backend Component	Core Technical Function	Runtime Behavior
Supabase access layer	Data persistence gateway	Reads/writes bots, documents, chunks, sessions, messages

Core Backend Working Model

The backend is designed around **stateful orchestration**. Incoming requests are not treated as isolated function calls; they are interpreted against bot configuration, ownership scope, usage limits, and session continuity.

For management operations, the backend validates ownership first, then checks platform limits before expensive work starts. This means upload, scrape, and crawl operations are capacity-aware from the start. Once input passes those gates, the backend converts raw content into chunks, generates embeddings, and stores them as retrieval-ready records.

For chat operations, the backend validates bot key and bot status, loads session state, loads recent history, and then decides how to answer:

1. If the bot has a flow graph, the flow engine is authoritative and advances node-by-node.
2. If no flow is present, it executes the direct retrieval-backed answer path.

This dual-path runtime is central to the architecture because it supports both guided scripted experiences and open retrieval-based QA without changing the client interface.

Data and State Topology Inside Backend

Stored Entity	Technical Purpose in Backend Runtime
Bot metadata	Defines bot identity, active state, prompt behavior, appearance, flow graph
Documents	Tracks source files/websites and ingestion metadata
Document chunks	Stores retrieval units and embeddings for semantic/lexical search
Chat sessions	Stores mutable runtime state like current flow node
Chat messages	Stores immutable conversation history for continuity and context

Stored Entity	Technical Purpose in Backend Runtime
Vector search function	Executes similarity matching for retrieval stage

Ingestion Pipeline Flow Diagram

```
Incoming Request
      |
      v
+-----------------------------+
| API Router Dispatch         |
| /health  /bot  /chat        |
+-----------------------------+
      |
      v
+-----------------------------+
| Access Control Gate         |
| Owner JWT OR Bot API Key    |
+-----------------------------+
      |
      +--------------------------------+
      |                                |
      v                                v
+-----------------------------+   +-----------------------------+
| Control Plane (Bot APIs)    |   | Runtime Plane (Chat API)    |
+-----------------------------+   +-----------------------------+
      |                                |
      v                                v
+-----------------------------+   +-----------------------------+
| Limits + Ownership Checks   |   | Session + History Load      |
+-----------------------------+   +-----------------------------+
      |                                |
      v                                v
+-----------------------------+   +-----------------------------+
| Ingestion Orchestration     |   | Flow Config Available?      |
| upload / scrape / crawl     |   +-----------------------------+
+-----------------------------+             | yes / no
      |                                     |
      v                                     v
+-----------------------------+   +-----------------------------+
| Text Extraction + Chunking  |   | Flow Executor OR            |
+-----------------------------+   | Retrieval-Response Pipeline |
      |                          +-----------------------------+
      v                                     |
+-----------------------------+             v
| Embedding Generation        |   +-----------------------------+
| microservice -> local fallback|  | Query Adaptation + Hybrid RAG|
+-----------------------------+   | dense + lexical + fusion    |
      |                          +-----------------------------+
      v                                     |
+-----------------------------+             v
| Persist Documents/Chunks/Vec |  +-----------------------------+
+-----------------------------+   | Response Generation         |
      |                          | Groq with key failover      |
      v                          +-----------------------------+
+-----------------------------+             |
| Return status/progress      |             v
+-----------------------------+   +-----------------------------+
                                  | Persist Messages + Session  |
                                  +-----------------------------+
                                             |
                                             v
                                  +-----------------------------+
                                  | Return Chat Response        |
                                  +-----------------------------+
```

Complete backend Flow

```
Incoming Request
      |
      v
+-------------------------+
| API Router Dispatch     |
| (health / bot / chat)   |
+-------------------------+
      |
      v
+-------------------------+
| Access Control Gate     |
| - Owner JWT OR          |
| - Bot API Key           |
+-------------------------+
      |
      +-----------------------------+
      |                             |
      v                             v
+-------------------------+   +-------------------------+
| Control Path (Bot APIs) |   | Runtime Path (Chat API) |
+-------------------------+   +-------------------------+
      |                             |
      v                             v
+-------------------------+   +-------------------------+
| Limit + Ownership Check |   | Load Session + History  |
+-------------------------+   +-------------------------+
      |                             |
      v                             v
+-------------------------+   +-------------------------+
| Ingestion Orchestration |   | Flow Exists?            |
| upload/scrape/crawl     |   +-------------------------+
+-------------------------+           | yes / no
      |                               |
      v                               v
+-------------------------+   +-------------------------+
| Text Extract + Chunking |   | Flow Executor OR        |
+-------------------------+   | Retrieval Pipeline      |
      |                       +-------------------------+
      v                               |
+-------------------------+           v
| Embedding Generation    |   +-------------------------+
| microservice -> fallback|   | Query Adaptation + RAG  |
+-------------------------+   | dense + lexical + fusion |
      |                       +-------------------------+
      v                               |
+-------------------------+           v
| Persist Docs/Chunks/Vec |   +-------------------------+
+-------------------------+   | Response Generation     |
      |                       | (Groq failover)         |
      v                       +-------------------------+
+-------------------------+           |
| Return status/progress  |           v
+-------------------------+   +-------------------------+
                              | Persist Messages/Session |
                              +-------------------------+
                                        |
                                        v
                              +-------------------------+
                              | Return Chat Response    |
                              +-------------------------+---
```

6. API Endpoints Summary

#	Method	Full Endpoint	Router Group	Auth Type	Request Body	Query Params	Primary Technical Function
1	GET	/	Root	None	None	None	Returns backend service metadata/status
2	GET	/health	Health	None	None	None	Health check for service availability
3	GET	/limits	Health	None	None	None	Returns public free-tier limits summary
4	GET	/user/usage	Health	User JWT	None	None	Returns current user usage metrics vs limits
5	POST	/bot/create	Bot	User JWT	BotCreate	None	Creates bot record and generates API key
6	GET	/bot/list	Bot	User JWT	None	None	Lists all bots owned by current user
7	GET	/bot/{bot_id}	Bot	User JWT	None	None	Fetches one bot by id after ownership check
8	PATCH	/bot/{bot_id}	Bot	User JWT	BotUpdate	None	Updates bot fields (prompt, flow, appearance, active state, etc.)
9	DELETE	/bot/{bot_id}	Bot	User JWT	None	None	Deletes bot and related documents/chunks/messages
10	GET	/bot/{bot_id}/storage	Bot	User JWT	None	None	Computes storage, chunk, doc, website usage for bot
11	POST	/bot/{bot_id}/upload	Bot	User JWT	multipart/form-data (file)	None	Uploads file, processes text, chunks content, generates/stores embeddings
12	GET	/bot/{bot_id}/documents	Bot	User JWT	None	None	Lists documents for a bot
13	DELETE	/bot/{bot_id}/documents/{document_id}	Bot	User JWT	None	None	Deletes one document and its associated chunks
14	POST	/bot/{bot_id}/train	Bot	User JWT	None	None	Generates embeddings for chunks where embedding is null
15	POST	/bot/{bot_id}/regenerate-key	Bot	User JWT	None	None	Rotates bot API key

#	Method	Full Endpoint	Router Group	Auth Type	Request Body	Query Params	Primary Technical Function
16	POST	/bot/{bot_id}/scrape	Bot	User JWT	ScrapeRequest	None	Scrapes one URL, chunks content, generates/stores embeddings
17	POST	/bot/{bot_id}/scrape-stream	Bot	User JWT	ScrapeRequest	None	Same as scrape, but with SSE progress stages
18	POST	/bot/{bot_id}/crawl-site	Bot	User JWT	CrawlRequest	None	Multi-page site crawling with SSE progress + chunk/embed pipeline
19	GET	/bot/{bot_id}/document/{document_id}/content	Bot	User JWT	None	None	Returns document metadata, ordered chunks, and combined full text
20	GET	/chat/{bot_id}/info	Chat	Bot API Key	None	None	Returns public bot runtime info (name + merged appearance settings)
21	POST	/chat/{bot_id}	Chat	Bot API Key	ChatRequest	None	Main chat execution path (flow executor or fallback RAG/LLM), persists messages
22	POST	/chat/{bot_id}/preview	Chat	User JWT	ChatRequest	None	Owner-only preview chat execution
23	DELETE	/chat/{bot_id}/sessions/{session_id}	Chat	Bot API Key	None	None	Deletes one session and its messages
24	DELETE	/chat/{bot_id}/sessions	Chat	Bot API Key	None	days_old (default 7)	Deletes sessions older than threshold and related messages
25	GET	/chat/{bot_id}/history/{session_id}	Chat	Bot API Key	None	None	Returns ordered message history for a session

7. Database Design & Schema

The platform uses a relational PostgreSQL design with vector search support through pgvector. The schema is built to support four backend goals at the same time: multi-tenant bot ownership, document-based knowledge ingestion, fast semantic retrieval, and session-based conversational continuity.

At a structural level, the design separates data into identity, configuration, knowledge, and conversation domains. Identity is handled through user profiles. Bot configuration stores prompts, API keys, flow definitions, and appearance settings. Knowledge is split into documents and chunks, where chunks carry embeddings for retrieval. Conversation is split into immutable message history and mutable session state.

Core Table Design

Table	Purpose	Primary Key	Main Foreign Keys
profiles	User profile extension for authenticated users	id	id -> auth.users.id
bots	Bot ownership, configuration, API key, flow and appearance settings	id	user_id -> profiles.id
documents	Source document/web metadata for each bot	id	bot_id -> bots.id
document_chunks	Chunk-level retrieval units with text and embedding vector	id	document_id -> documents.id, bot_id -> bots.id
chat_messages	Ordered conversation messages by bot and session	id	bot_id -> bots.id
chat_sessions	Current session runtime state for flow execution	id	bot_id -> bots.id, unique(session_id, bot_id)

Relationship Model

Relationship	Type	Meaning
Profile -> Bots	One-to-many	One user can own multiple bots
Bot -> Documents	One-to-many	One bot can have multiple source entries
Document -> Chunks	One-to-many	One document is split into many chunks
Bot -> Chunks	One-to-many	Retrieval corpus is scoped per bot
Bot + Session -> Messages	One-to-many	Each session accumulates many messages
Bot + Session -> Session State	One-to-one per pair	One mutable session state record per bot-session

Retrieval-Oriented Design

Retrieval Component	Implementation
Vector storage	embedding VECTOR(384) in document_chunks
Vector index type	IVFFlat with vector_cosine_ops
Retrieval function	match_documents(query_embedding, match_count, filter_bot_id)
Returned data	id, content, similarity
Similarity expression	1 - (embedding <=> query_embedding)

8. Chunking Size

Chunking is implemented as a token-aware preprocessing stage in the backend and is controlled by two configuration parameters: chunk_size and chunk_overlap. In the current setup, chunk_size is 500 and chunk_overlap is 50. The system first extracts plain text from uploaded or scraped sources, then divides the text into chunks that are suitable for retrieval and embedding generation. This is done to ensure that each chunk remains semantically meaningful while still being small enough for efficient vector search.

The chunking logic is hierarchical. It starts with paragraph-level splitting (double newline boundaries), then appends paragraphs into the current chunk until the token threshold is reached. If adding the next paragraph exceeds the chunk limit, the current chunk is finalized and a new chunk is started. To preserve continuity, overlap is added from the end of the previous chunk. In this implementation, the overlap is created from the last 50 words of the previous chunk, which helps reduce context breakage across chunk boundaries. When a single paragraph itself is too large, the algorithm falls back to sentence-level splitting so that large text blocks can still be chunked safely.

Token counting is performed using tiktoken (cl100k_base) for more accurate chunk sizing. If tokenization fails for any reason, the system uses a fallback approximation based on character length divided by four. This fallback prevents ingestion failure and keeps the pipeline resilient.

Chunking is tightly integrated with platform limits. After chunk generation, the backend checks whether adding the new chunks would exceed the per-bot cap (max_chunks_per_bot = 200). If the limit would be exceeded, ingestion is rejected with a clear error. This makes chunking not only a retrieval-quality

mechanism, but also a capacity-control mechanism that protects database growth and free-tier operational stability.

9. Embedding Microservice

The embedding microservice is the vector-generation engine of the system. Theoretically, its role is to convert natural language text into dense numerical vectors so semantic similarity can be computed during retrieval. In this architecture, retrieval quality depends on this representation layer because both documents and user queries are mapped into the same vector space before matching.

From a systems-design perspective, separating embeddings into a dedicated service gives clean isolation between orchestration logic and model inference logic. The main backend handles bot management, ingestion flow, retrieval orchestration, and chat runtime, while the embedding service focuses only on fast and consistent vector generation. This reduces memory pressure on the main API and allows independent scaling and deployment.

In this project, the microservice is implemented with FastAPI and uses the sentence-transformers model all-MiniLM-L6-v2, producing 384-dimensional vectors. The model is loaded through a cached lazy loader and also preloaded at startup, so the service can avoid repeated model initialization overhead. The service exposes health/readiness endpoints and two inference endpoints: single text embedding and batch embedding.

Theoretical Role in RAG Pipeline

In theory, retrieval-augmented generation depends on three mappings:

1. Source content -> chunk vectors
2. User query -> query vector
3. Similarity search(query vector, chunk vectors) -> relevant context

The embedding microservice is responsible for the first two mappings. Without stable embeddings, dense retrieval quality drops, and downstream generation gets weaker context.

Project Implementation Summary

Area	Implementation in this project
Service Type	Standalone FastAPI microservice
Core Model	all-MiniLM-L6-v2
Vector Dimension	384
Single Inference Endpoint	POST /embed
Batch Inference Endpoint	POST /embed/batch
Health Endpoint	GET / and GET /health
Readiness Endpoint	GET /ready

Endpoint Contract Table

Endpoint	Method	Input	Output	Validation
/health	GET	None	status, model, dimension, ready	Lightweight health status
/ready	GET	None	readiness with model status	Confirms model load success
/embed	POST	text (string)	embedding (float array), dimension, model	Rejects empty text
/embed/batch	POST	texts (array of strings)	embeddings, dimension, model, count	Rejects empty list, all-empty list, >100 batch size

Runtime Integration with Main Backend

During ingestion and query handling, the backend calls the microservice first. If the microservice call fails (network error, timeout, unavailable service), backend

logic falls back to local embedding generation using the same model family. This gives graceful degradation instead of total pipeline failure.

Step	Backend Behavior
1	Try HTTP call to embedding microservice (/embed)
2	If successful, return embedding to caller
3	If failed, log microservice failure
4	Attempt local embedding generation fallback
5	If fallback also fails, raise error to caller

Why this microservice architecture is important

This design improves operational reliability and modularity. It keeps heavy ML inference outside the main backend process, supports independent deployment and scaling, and preserves continuity through fallback logic. In practical terms, it helps maintain stable chat and ingestion behavior even when one inference path is temporarily unavailable.

10. Main Core Libraries Used in Backend, NLP, and NLU

The backend is built as an API orchestration system with specialized libraries for routing, authentication, ingestion, retrieval, NLP/NLU preprocessing, and response generation. The table below lists the main libraries, what they do in this project, and why they are used.

A) Core Backend Runtime Libraries

Library	Purpose in Project	Why it is Used
FastAPI	Builds all backend endpoints for health, bot management, and chat runtime	High-performance async API framework with clean routing and validation integration

Library	Purpose in Project	Why it is Used
Uvicorn	Runs the FastAPI ASGI server	Lightweight production-ready server for async Python APIs
Pydantic	Defines request and response schemas	Strong data validation and safer API contracts
Pydantic Settings	Loads environment variables into typed settings	Centralized configuration management for keys, URLs, limits, and runtime switches
Supabase Python SDK	Performs all database CRUD and RPC operations	Simplifies PostgreSQL access and integrates well with Supabase auth/data model
Httpx	Makes async HTTP calls from backend to embedding microservice	Non-blocking network calls for inference services
Python Multipart	Supports file upload handling in API endpoints	Required for multipart form-data ingestion

B) NLP and NLU Libraries

Library	Purpose in Project	Why it is Used
spaCy	Token normalization, lemmatization, and entity extraction in NLU preprocessing	Improves query normalization and entity handling before retrieval
YAKE	Keyword extraction fallback/feature in NLU	Lightweight unsupervised keyword extraction for retrieval signal enhancement
Transformers	Zero-shot intent classification pipeline fallback	Helps classify query intent when available

Library	Purpose in Project	Why it is Used
scikit-learn	Provides English stopword set used in token filtering	Improves keyword quality by removing low-information tokens
Groq SDK	Used in NLU intent LLM path and final response generation path	Fast LLM inference and failover-capable generation in current runtime

C) Retrieval and Ranking Libraries

Library	Purpose in Project	Why it is Used
NumPy	Vector math and cosine similarity fallback operations	Efficient numeric operations for similarity scoring
rank-bm25	Sparse lexical retrieval scoring	Complements dense vector search for exact-term relevance
PostgreSQL pgvector	Stores and searches embedding vectors	Enables semantic nearest-neighbor retrieval inside database

D) Embedding Libraries

Library	Purpose in Project	Why it is Used
sentence-transformers	Generates local fallback embeddings and powers embedding microservice model	Reliable sentence-level embedding generation for RAG
Torch	Runtime backend for sentence-transformers model inference	Required model execution engine

Library	Purpose in Project	Why it is Used
all-MiniLM-L6-v2 model	Embedding model used (384 dimensions)	Good quality/speed/storage balance for chatbot retrieval

E) Document Processing Libraries

Library	Purpose in Project	Why it is Used
pypdf	Extracts text from PDF uploads	Supports common enterprise document format
python-docx	Extracts text from DOCX files	Adds Microsoft Word ingestion support
tiktoken	Token-aware chunk sizing	Creates more retrieval-stable chunks than plain character splitting

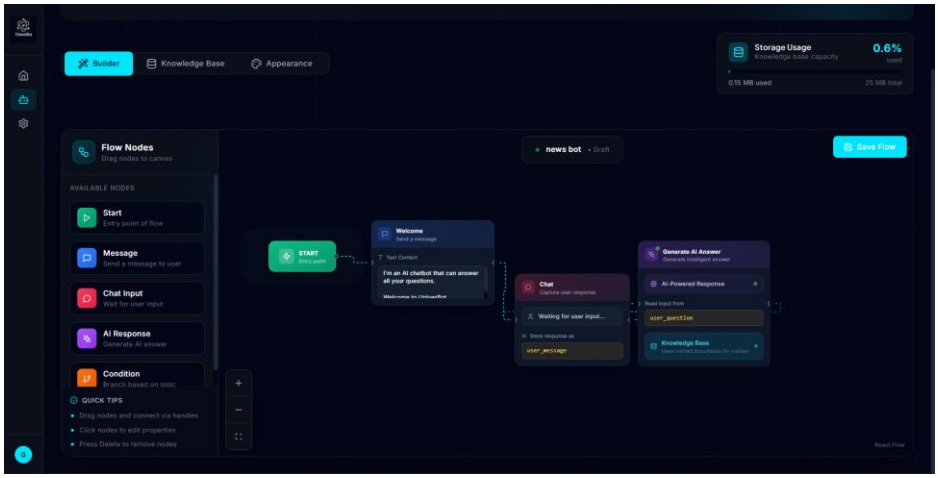
F) Web Scraping and Crawling Libraries

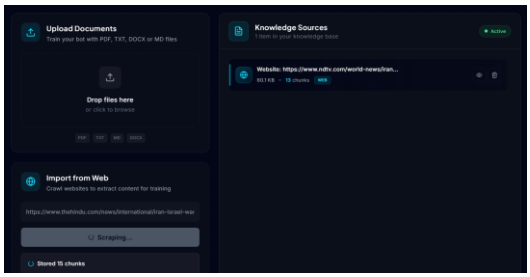
Library	Purpose in Project	Why it is Used
Crawl4AI	Main scraping and crawling engine	Better extraction quality for modern web pages
Playwright	Browser automation backend for Crawl4AI	Handles JavaScript-rendered websites
nest-asyncio	Event loop compatibility support on specific environments	Improves stability in mixed async execution contexts

G) Authentication and Security-Related Libraries

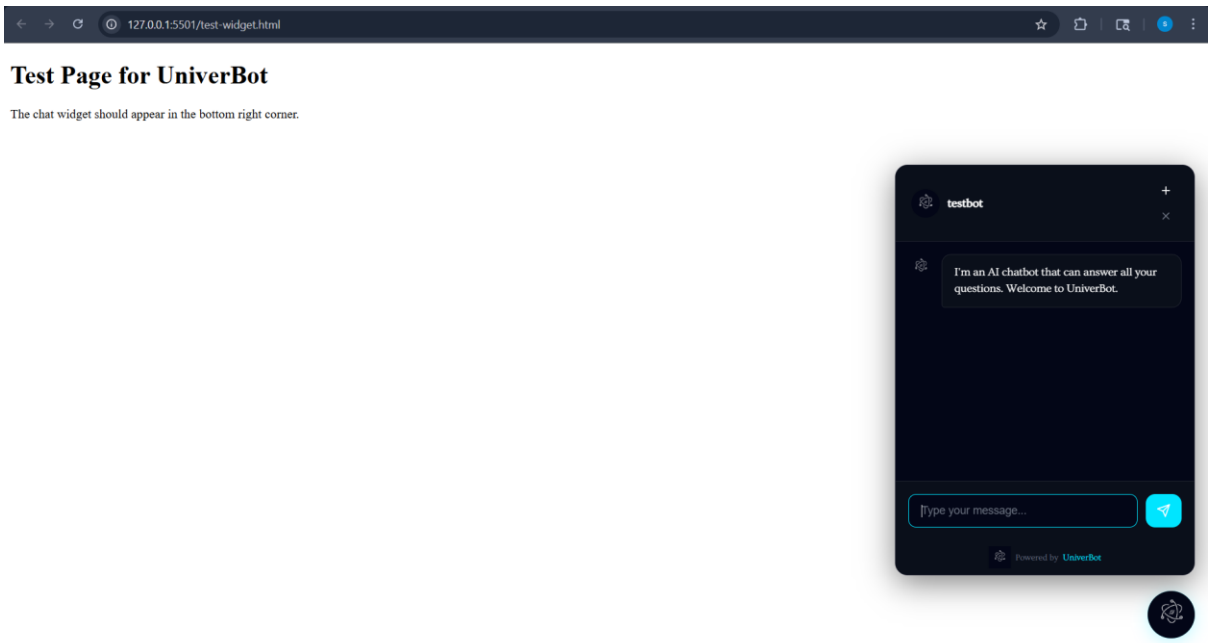
Library	Purpose in Project	Why it is Used
Supabase Auth integration	User token verification and identity context	Provides owner-level access control
FastAPI security utilities	Bearer auth handling for API key and JWT guards	Enforces route-level security boundarie

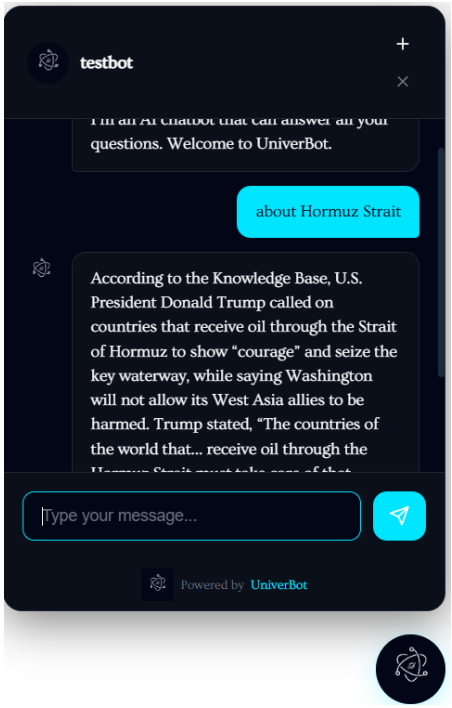
11. Results





Bot Configuration
news bot
about news
ID: e08d3271...
API Key: ub_AinXhZI69...


127.0.0.1:5501/test-widget.html
Test Page for UniverBot
The chat widget should appear in the bottom right corner.
testbot
I'm an AI chatbot that can answer all your questions. Welcome to UniverBot.
Type your message...
Powered by UniverBot


testbot
questions. Welcome to UniverBot.
about Hormuz Strait
According to the Knowledge Base, U.S. President Donald Trump called on countries that receive oil through the Strait of Hormuz to show "courage" and seize the key waterway, while saying Washington will not allow its West Asia allies to be harmed. Trump stated, "The countries of the world that... receive oil through the
Type your message...
Powered by UniverBot